

```

# 🎮 Game Recommendation System + Visualization (Ready to Run)

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 1 Built-in Dataset
data = {
    'Game': [
        'Battle Quest', 'Puzzle Master', 'Space Explorer', 'Word Wizard', 'Soccer Stars',
        'Racing Rivals', 'Treasure Hunt', 'Zombie Attack', 'Farm Builder', 'Chess Pro'
    ],
    'Genre': [
        'Action', 'Puzzle', 'Adventure', 'Puzzle', 'Sports',
        'Racing', 'Adventure', 'Action', 'Strategy', 'Strategy'
    ],
    'Platform': [
        'PC', 'Mobile', 'PC', 'Mobile', 'Console',
        'Mobile', 'PC', 'Console', 'Mobile', 'PC'
    ],
    'Difficulty': [
        'Hard', 'Easy', 'Medium', 'Medium', 'Medium',
        'Hard', 'Easy', 'Hard', 'Easy', 'Hard'
    ],
    'Rating': [4.5, 4.2, 4.0, 4.3, 4.4, 4.6, 4.1, 4.0, 4.3, 4.8],
    'Recommended_For': [
        'Teens', 'Kids', 'Everyone', 'Kids', 'Everyone',
        'Teens', 'Everyone', 'Teens', 'Kids', 'Everyone'
    ]
}
df = pd.DataFrame(data)
display(df.head())

# 2 Visualization
plt.figure(figsize=(8,4))
df['Genre'].value_counts().plot(kind='bar', title='Number of Games per Genre')
plt.xlabel('Genre'); plt.ylabel('Count')
plt.show()

plt.figure(figsize=(8,4))
plt.bar(df['Game'], df['Rating'])
plt.xticks(rotation=45, ha='right')
plt.title('Game Ratings Comparison')
plt.ylabel('Rating')
plt.show()

# 3 Recommendation System
df['combined'] = df['Genre'] + ' ' + df['Platform'] + ' ' + df['Difficulty'] + ' ' + df['Recommended_For']
vectorizer = CountVectorizer()
feature_matrix = vectorizer.fit_transform(df['combined'])
similarity = cosine_similarity(feature_matrix)

def recommend(game_name, top_n=3):
    if game_name not in df['Game'].values:
        return f"Game '{game_name}' not found."
    idx = df[df['Game'] == game_name].index[0]
    scores = list(enumerate(similarity[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)[1:top_n+1]
    results = df.iloc[[i for i, _ in scores]][['Game', 'Genre', 'Rating']]
    return results

print("🎮 Recommendation Example for 'Battle Quest':")
display(recommend('Battle Quest'))

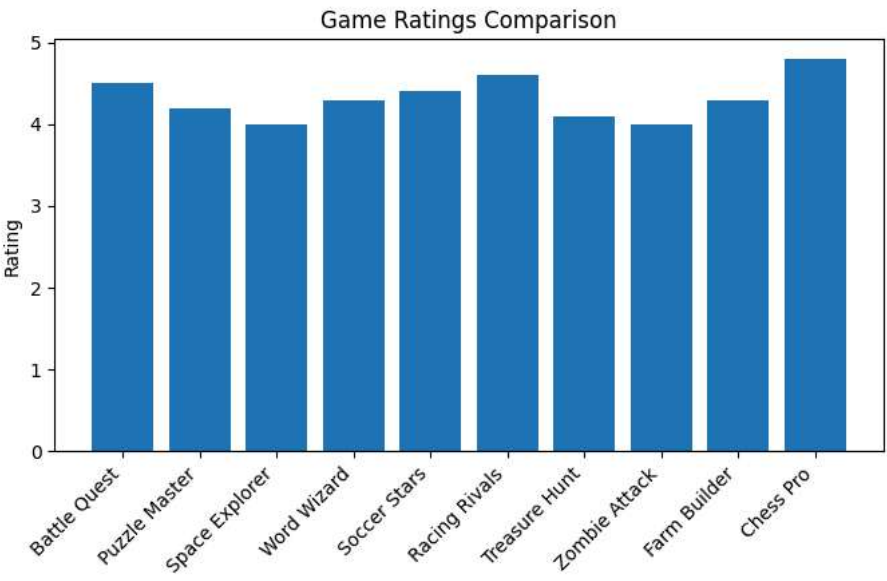
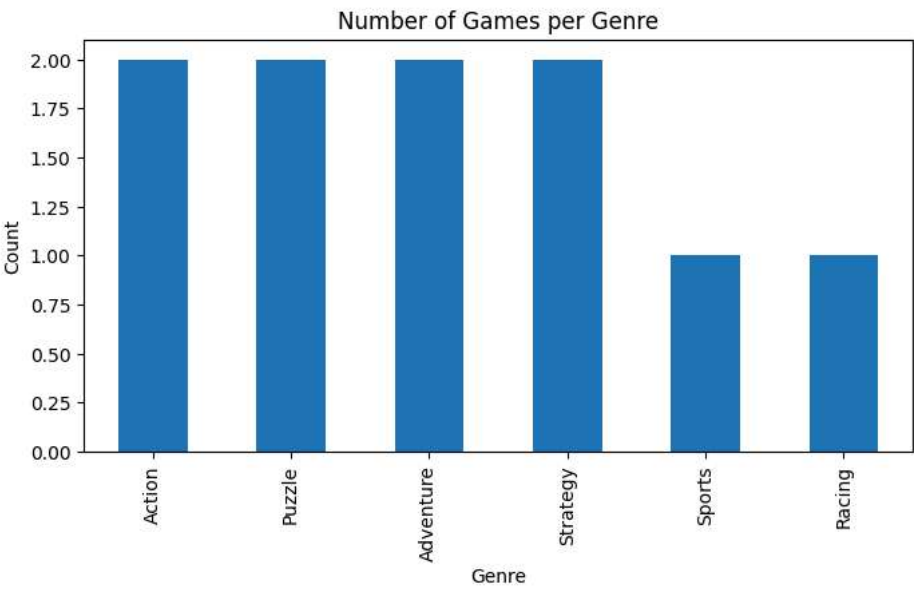
# 4 Simple AI Agent
def agent_suggest(preferences):
    subset = df.copy()
    for key, val in preferences.items():
        if key in subset.columns and val:
            subset = subset[subset[key].str.lower() == val.lower()]
    if subset.empty:
        best = df.sort_values('Rating', ascending=False).iloc[0]
        return {'Suggested_Game': best['Game'], 'Reason': 'No exact match, top-rated chosen.'}
    best = subset.sort_values('Rating', ascending=False).iloc[0]

```

```
return {'Suggested_Game': best['Game'], 'Reason': 'Best match with your preferences.'}

print("🤖 Agent Suggestion Example:")
print(agent_suggest({'Genre': 'Strategy', 'Platform': 'PC'}))
```

	Game	Genre	Platform	Difficulty	Rating	Recommended_For
0	Battle Quest	Action	PC	Hard	4.5	Teens
1	Puzzle Master	Puzzle	Mobile	Easy	4.2	Kids
2	Space Explorer	Adventure	PC	Medium	4.0	Everyone
3	Word Wizard	Puzzle	Mobile	Medium	4.3	Kids
4	Soccer Stars	Sports	Console	Medium	4.4	Everyone



🤖 Recommendation Example for 'Battle Quest':

	Game	Genre	Rating
7	Zombie Attack	Action	4.0
5	Racing Rivals	Racing	4.6
9	Chess Pro	Strategy	4.8

```
🤖 Agent Suggestion Example:
{'Suggested_Game': 'Chess Pro', 'Reason': 'Best match with your preferences.'}
```

